

UML DIAGRAMS

Comprehensive Reference Document
10 Systems · 20 Diagrams

Systems Covered

1. *Course Registration System*
2. *Student Marks Analysis System*
3. *Expert System - Medicine Prescription*
4. *Remote Diagnostics System*
5. *Hospital Management System*
6. *ATM Banking System*
7. *UPI Payment Application*
8. *Online Shopping Portal*
9. *Courier Management System*
10. *Tourism Management System*

Software Engineering - UML Design Reference

1. Course Registration System

A university Course Registration System enables students to browse, register for, and drop courses. Administrators manage course offerings and generate reports. Professors view enrolled students and upload grades.

1.1 Use Case Diagram

Captures all functional interactions between Student, Admin, and Professor actors and the system boundary.

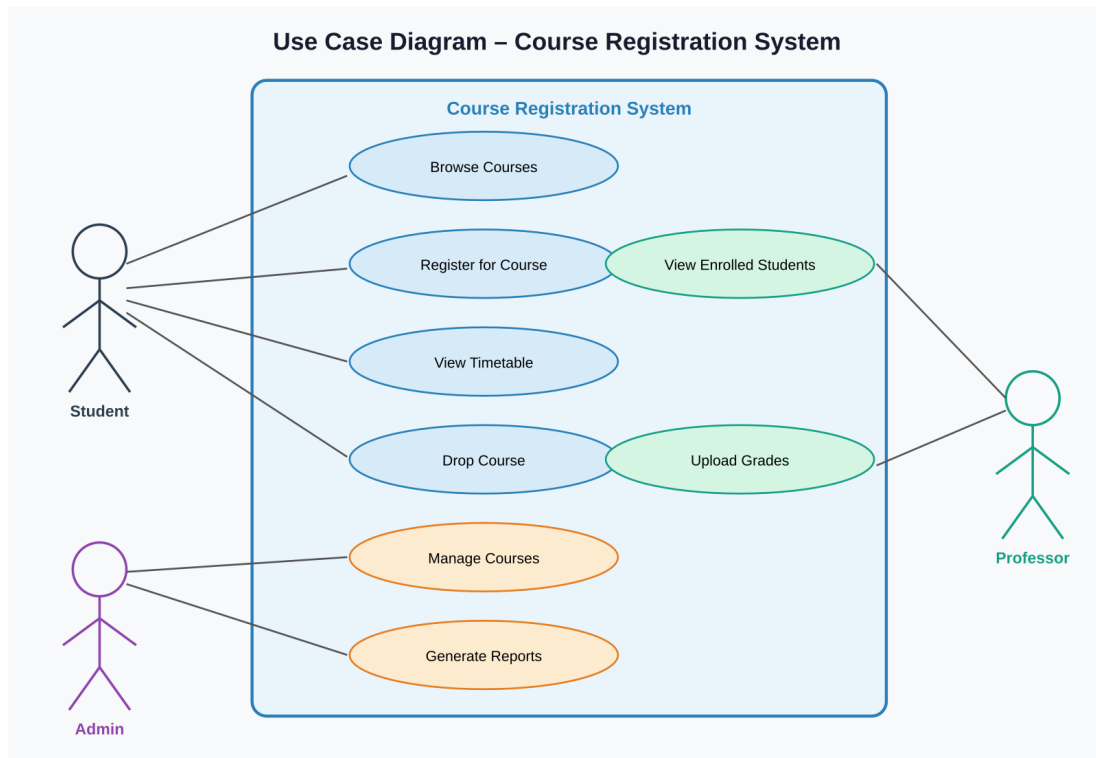


Figure 1.1 - Use Case Diagram: Course Registration System

Actors	Student, Admin, Professor
Key Use Cases	Browse Courses, Register for Course, View Timetable, Drop Course, Manage Courses, Generate Reports, View Enrolled Students, Upload Grades
Relationships	Association (Actor to Use Case); Include for prerequisite checks

1.2 Class Diagram

Models the static structure - six core entities with attributes, methods, and inter-relationships.

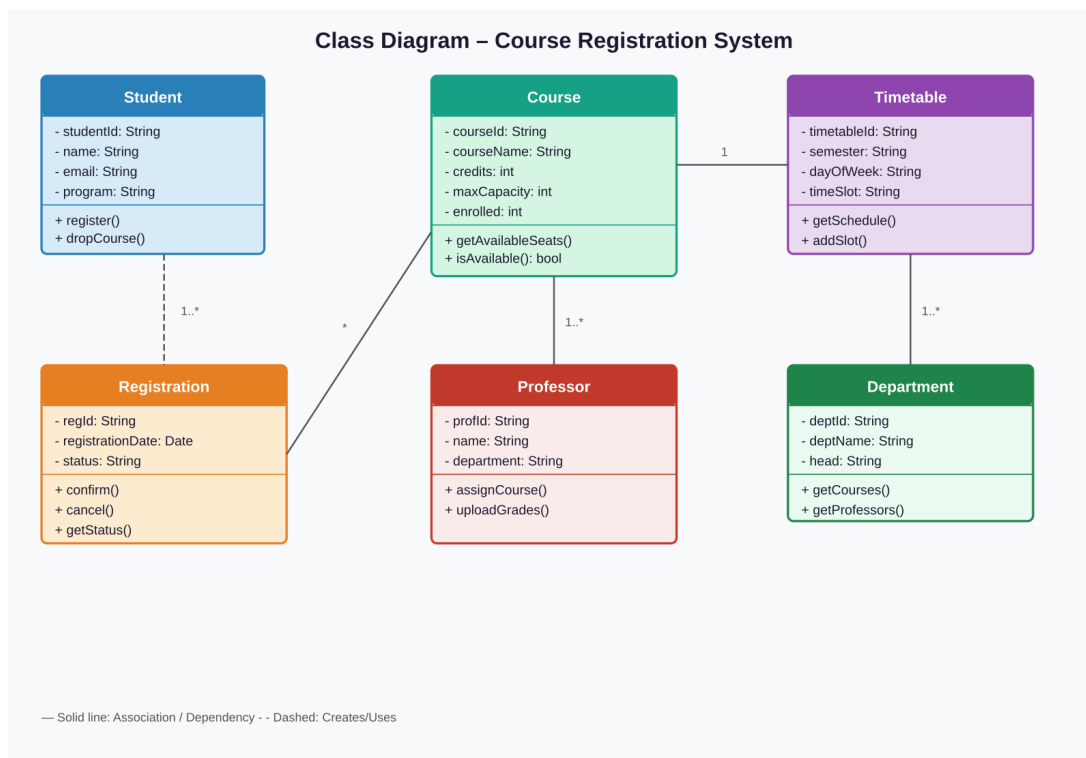


Figure 1.2 - Class Diagram: Course Registration System

Core Classes	Student, Course, Registration, Professor, Timetable, Department
Key Relationships	Student creates Registration (1..*); Professor teaches Course (1..*); Course has Timetable (1)
Design Pattern	Association + Dependency between Registration and Course

2. Student Marks Analysis System

The Student Marks Analysis System collects marks entered by examiners, processes them through an analysis engine, and generates detailed performance reports including grades, CGPA, and class rankings.

2.1 Class Diagram

Entities capture the full data model: students, subjects, marks records, analysis logic, generated reports, and examiners.

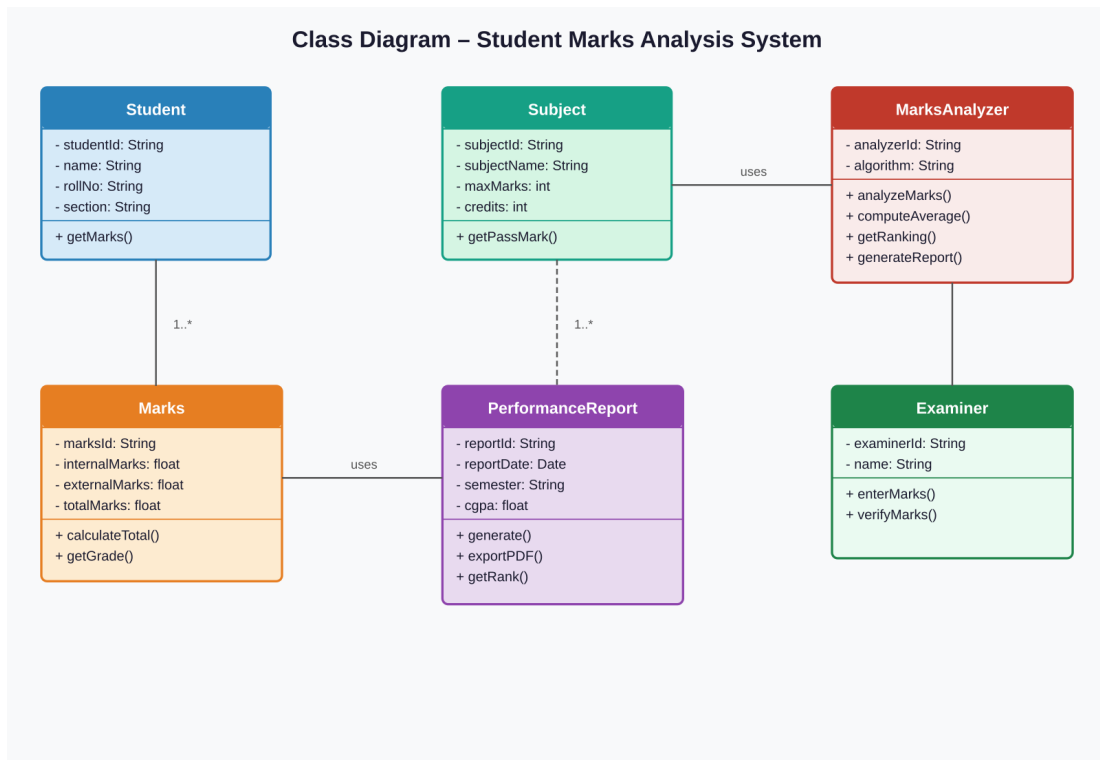


Figure 2.1 - Class Diagram: Student Marks Analysis System

Core Classes	Student, Subject, Marks, PerformanceReport, MarksAnalyzer, Examiner
Relationships	Student has Marks (1..*); MarksAnalyzer uses Marks data; MarksAnalyzer generates PerformanceReport
Key Methods	calculateTotal(), getGrade(), analyzeMarks(), computeAverage(), getRanking(), generate(), exportPDF()

2.2 Sequence Diagram

Shows chronological message flow between Examiner, System, MarksDB, Analyzer, and Report components.

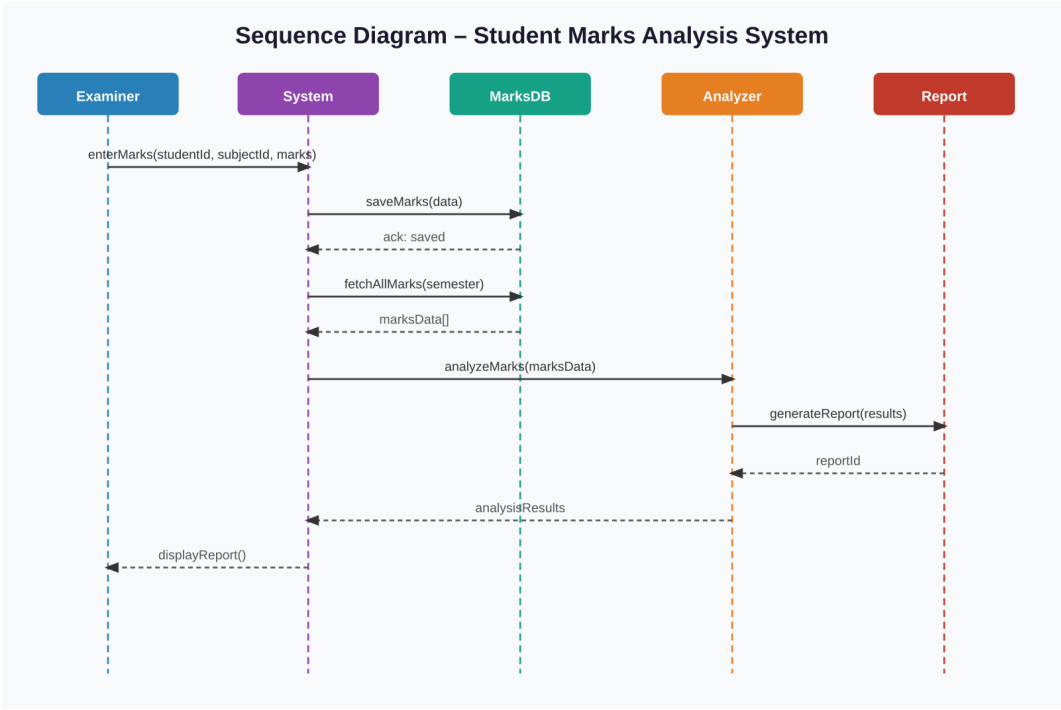


Figure 2.2 - Sequence Diagram: Student Marks Analysis System

Lifelines	Examiner > System > MarksDB > Analyzer > Report
Key Interactions	enterMarks() > saveMarks() > fetchAllMarks() > analyzeMarks() > generateReport() > displayReport()
Return Messages	Dashed arrows: ack, marksData[], reportId, analysisResults

3. Expert System - Medicine Prescription

An AI-driven Expert System that collects patient symptoms, matches them against a knowledge base, diagnoses probable diseases, and prescribes medicines. Doctors can review and update the knowledge base.

3.1 Use Case Diagram

Two actors (Patient and Doctor) interact with the Expert System boundary. Include relationship: Prescribe Medicine requires prior Diagnose Disease.

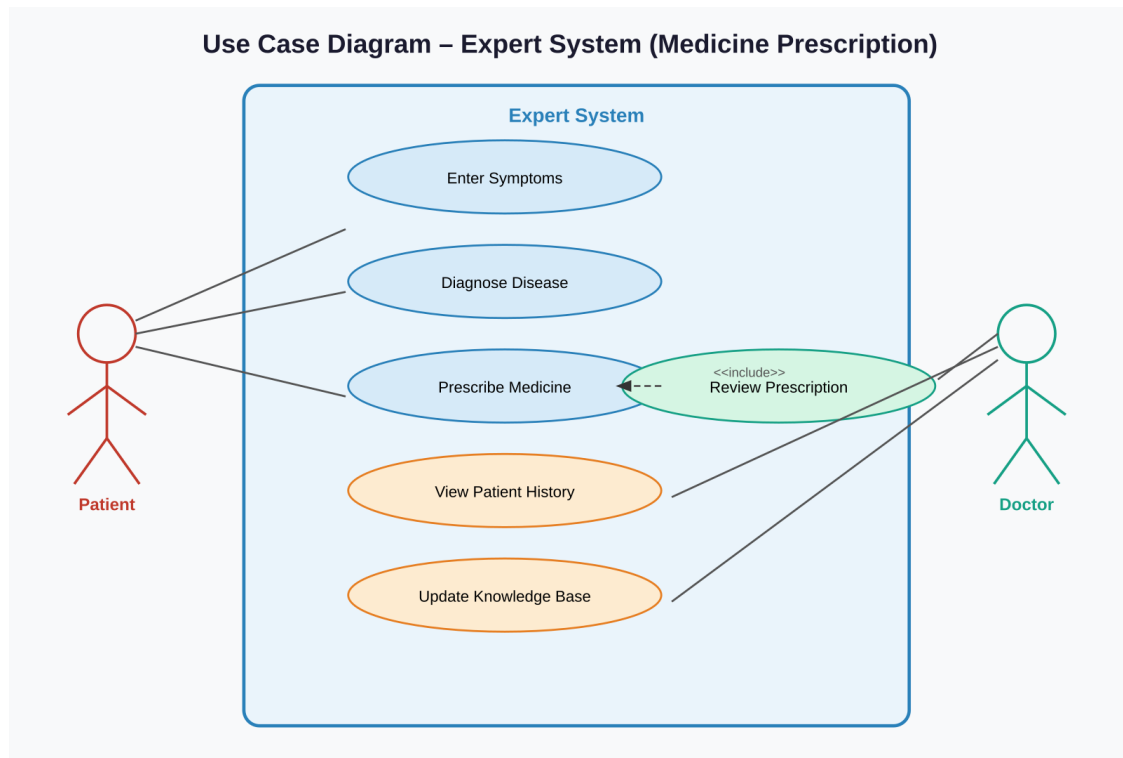


Figure 3.1 - Use Case Diagram: Expert System (Medicine Prescription)

Actors	Patient, Doctor
Key Use Cases	Enter Symptoms, Diagnose Disease, Prescribe Medicine, View Patient History, Update Knowledge Base, Review Prescription
Relationships	Association; Include between Prescribe Medicine and Diagnose Disease

3.2 Activity Diagram

Workflow: symptom entry > validation > knowledge-base matching > diagnosis confidence check > prescription or referral.

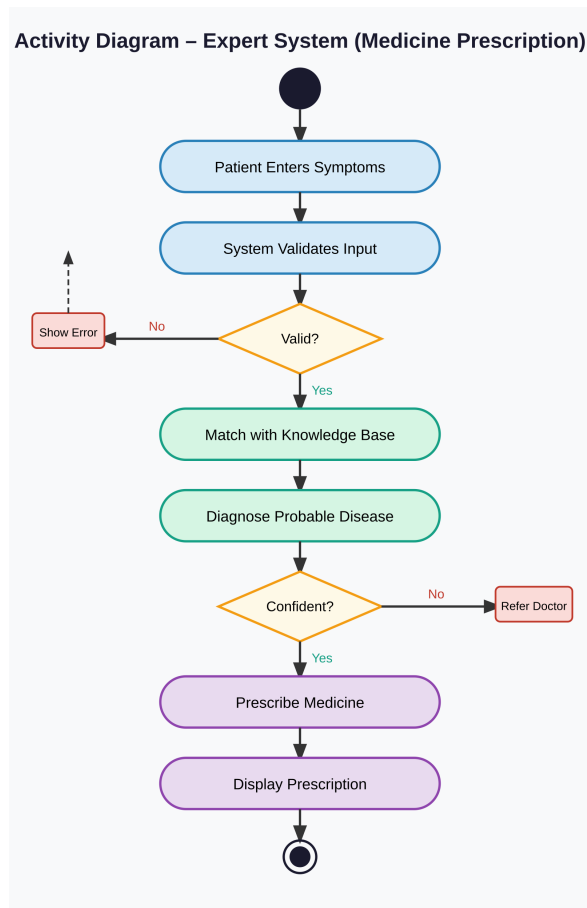


Figure 3.2 - Activity Diagram: Expert System (Medicine Prescription)

Decision Points	Valid Input?, Confident Diagnosis?
Alternate Flow	Invalid input loops back; Low confidence routes to Refer Doctor
End States	Display Prescription (happy path), Refer Doctor (uncertain path)

4. Remote Diagnostics System

An online doctor consultation platform allowing patients to log in, book video consultations, interact with doctors in real time, and receive digital prescriptions stored in the EHR.

4.1 Sequence Diagram

Message flow across Patient, App/Portal, Auth Server, Video Service, Doctor, and EHR Database during a consultation.

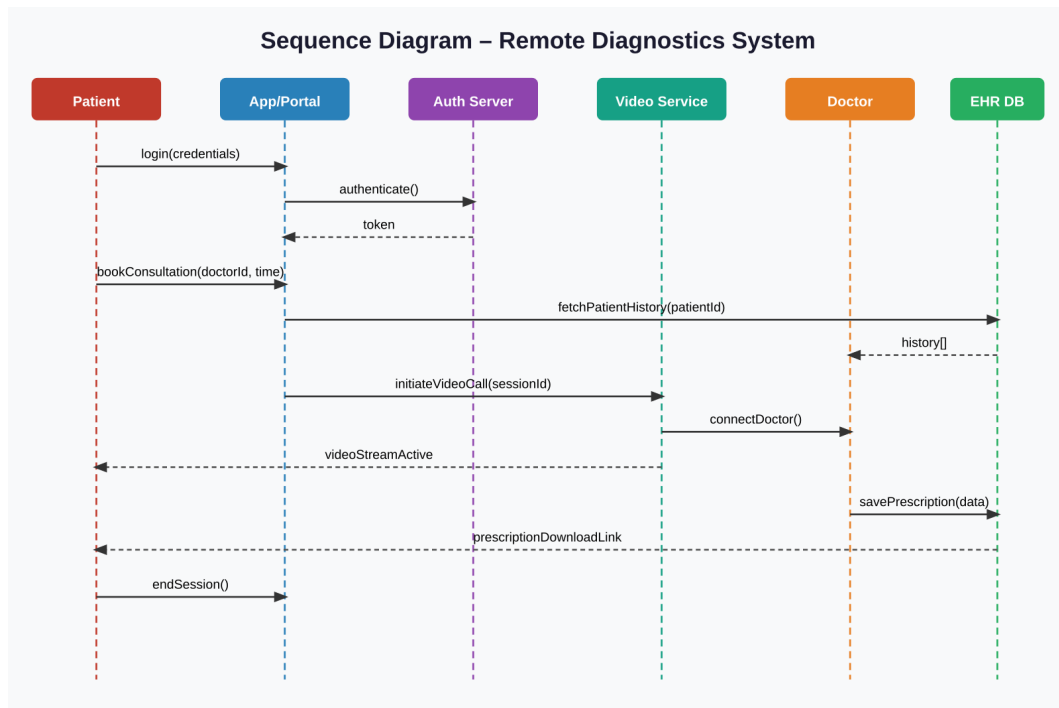


Figure 4.1 - Sequence Diagram: Remote Diagnostics System

Lifelines	Patient, App/Portal, Auth Server, Video Service, Doctor, EHR DB
Key Interactions	login > authenticate > bookConsultation > fetchHistory > initiateVideoCall > connectDoctor > savePrescription > endSession
Security	JWT token-based authentication via Auth Server

4.2 Deployment Diagram

Maps physical/logical nodes with communication protocols: Patient Device, Doctor Device, API Gateway, App Server, Video Server, Database Server.

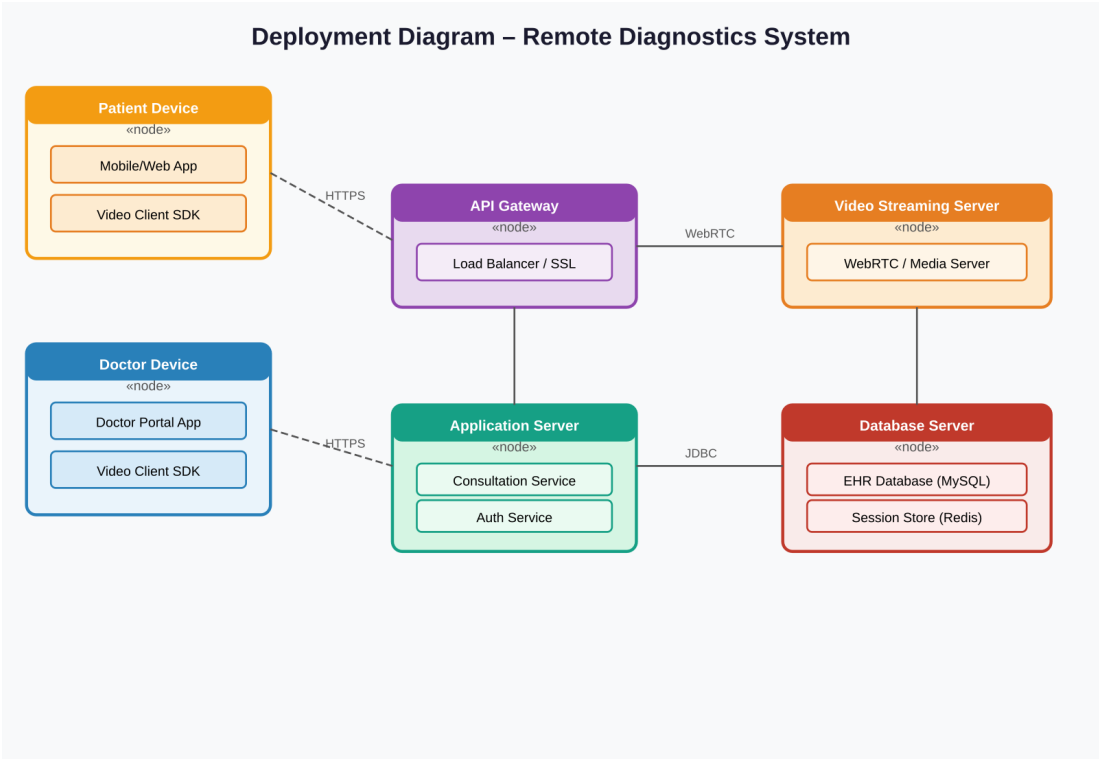


Figure 4.2 - Deployment Diagram: Remote Diagnostics System

Nodes	Patient Device, Doctor Device, API Gateway, Application Server, Video Streaming Server, Database Server
Components	Mobile/Web App, WebRTC Media Server, Consultation Service, Auth Service, EHR DB (MySQL), Session Store (Redis)
Protocols	HTTPS (client-gateway), WebRTC (video), JDBC (app-DB)

5. Hospital Management System

A comprehensive system managing patients, doctors, appointments, prescriptions, medical records, billing, ward allocation, and inventory across a hospital.

5.1 Use Case Diagram

Three primary actors - Patient, Doctor, and Admin - each with distinct use cases covering the full hospital workflow.

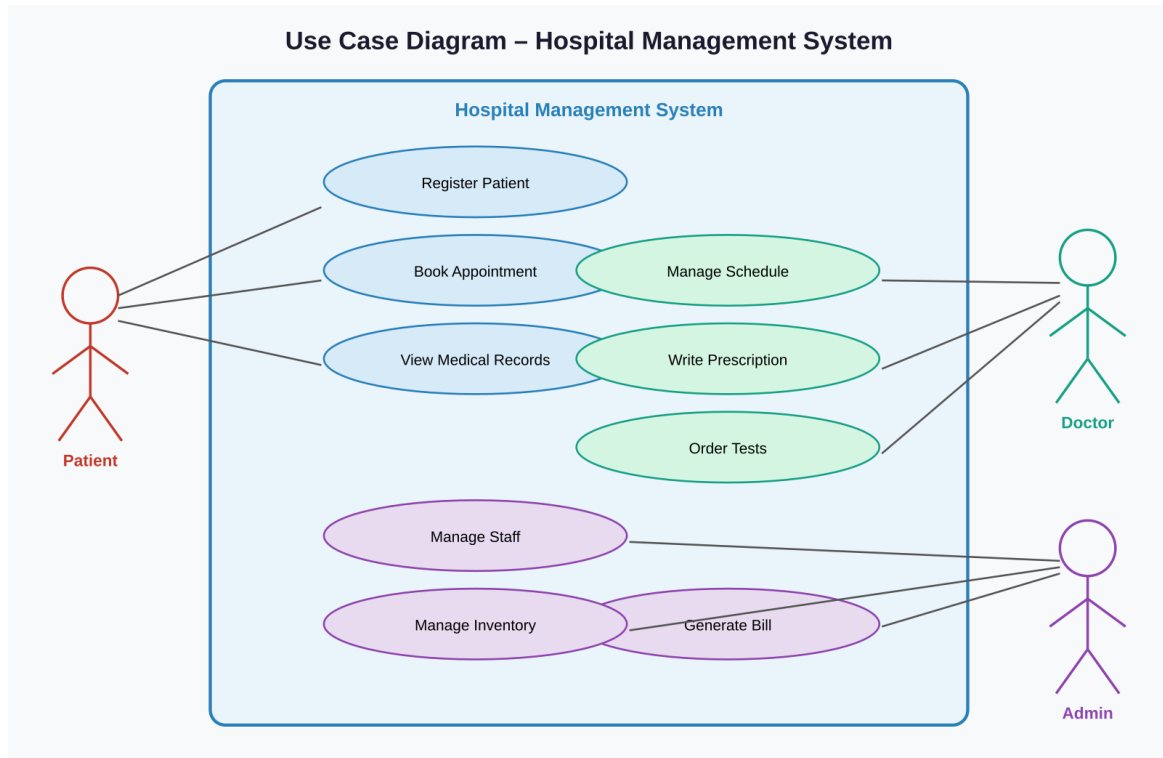


Figure 5.1 - Use Case Diagram: Hospital Management System

Actors	Patient, Doctor, Admin
Patient Use Cases	Register Patient, Book Appointment, View Medical Records
Doctor Use Cases	Manage Schedule, Write Prescription, Order Tests
Admin Use Cases	Manage Staff, Generate Bill, Manage Inventory

5.2 Class Diagram

Seven core classes with attributes and methods modelling the complete HMS data structure.

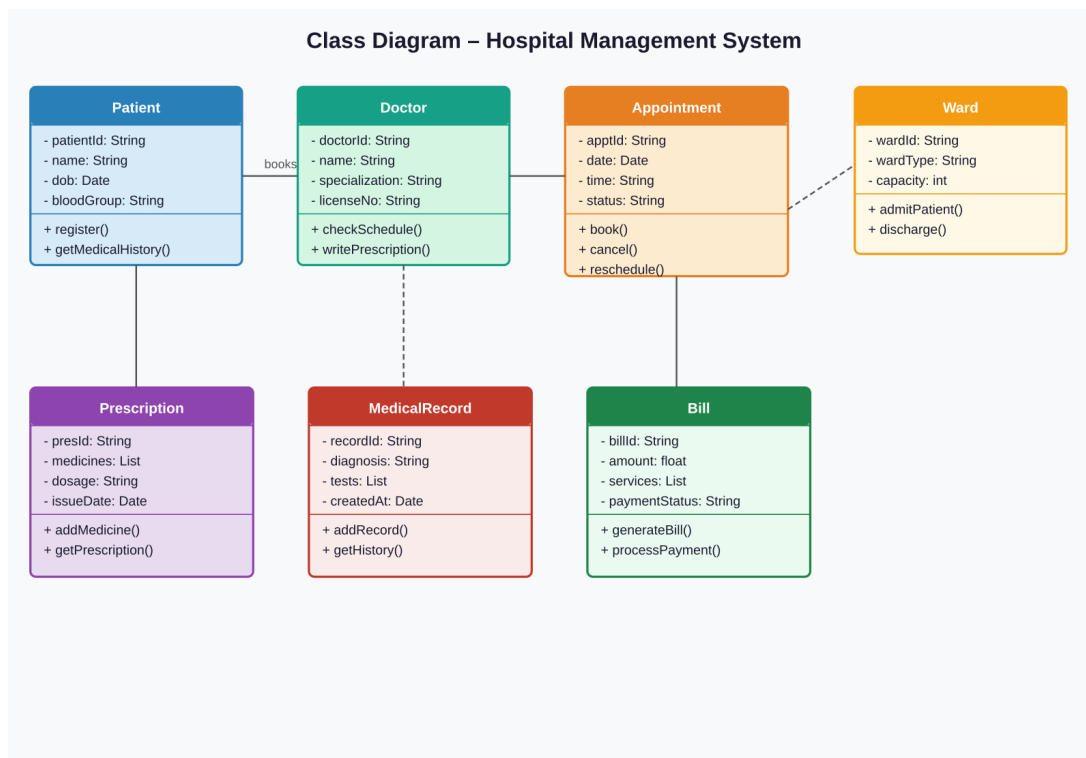


Figure 5.2 - Class Diagram: Hospital Management System

Core Classes	Patient, Doctor, Appointment, Prescription, MedicalRecord, Bill, Ward
Relationships	Patient books Appointment; Doctor writes Prescription/MedicalRecord; Bill generated from Appointment
Key Methods	register(), getMedicalHistory(), checkSchedule(), writePrescription(), book(), cancel(), generateBill(), processPayment()

6. ATM Banking System

An ATM system supporting card authentication and PIN verification, with three core transactions: Cash Withdrawal, Cash Deposit, and Balance Inquiry, with receipt printing.

6.1 Sequence Diagram

Full message flow between Customer, ATM UI, ATM Controller, Bank Server, and Account DB across authentication and all transaction types.

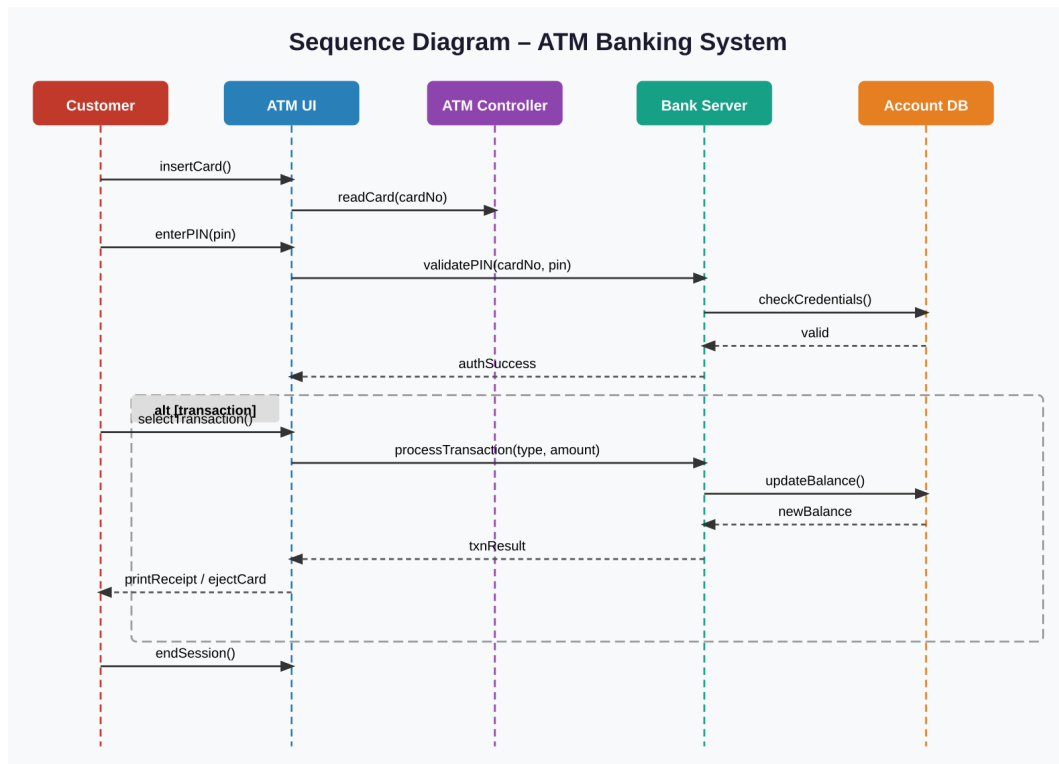


Figure 6.1 - Sequence Diagram: ATM Banking System

Lifelines	Customer, ATM UI, ATM Controller, Bank Server, Account DB
Key Interactions	insertCard > readCard > enterPIN > validatePIN > checkCredentials > selectTransaction > processTransaction > updateBalance > printReceipt
Alt Frame	alt[transaction] block covers Withdraw, Deposit, Balance Inquiry

6.2 Activity Diagram

Workflow from card insertion through PIN validation, three-way transaction branch, processing, receipt, and card ejection.

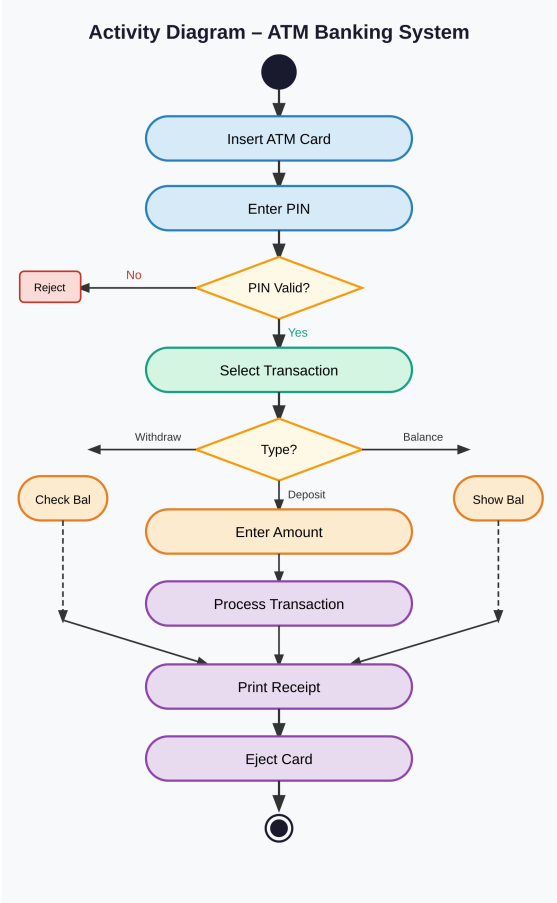


Figure 6.2 - Activity Diagram: ATM Banking System

Decision Points	PIN Valid?, Transaction Type?
Three Branches	Withdraw (check balance > dispense cash), Deposit (accept cash), Balance Inquiry (display balance)
Error Flow	Invalid PIN > Card Rejected and ejected

7. UPI Payment Application

A mobile-first UPI application enabling peer-to-peer fund transfers, bill payments, and scheduled payments via Virtual Payment Addresses (VPA) linked to bank accounts.

7.1 Class Diagram

Seven classes model users, bank accounts, transactions, VPAs, notifications, bill payments, and the UPI gateway.

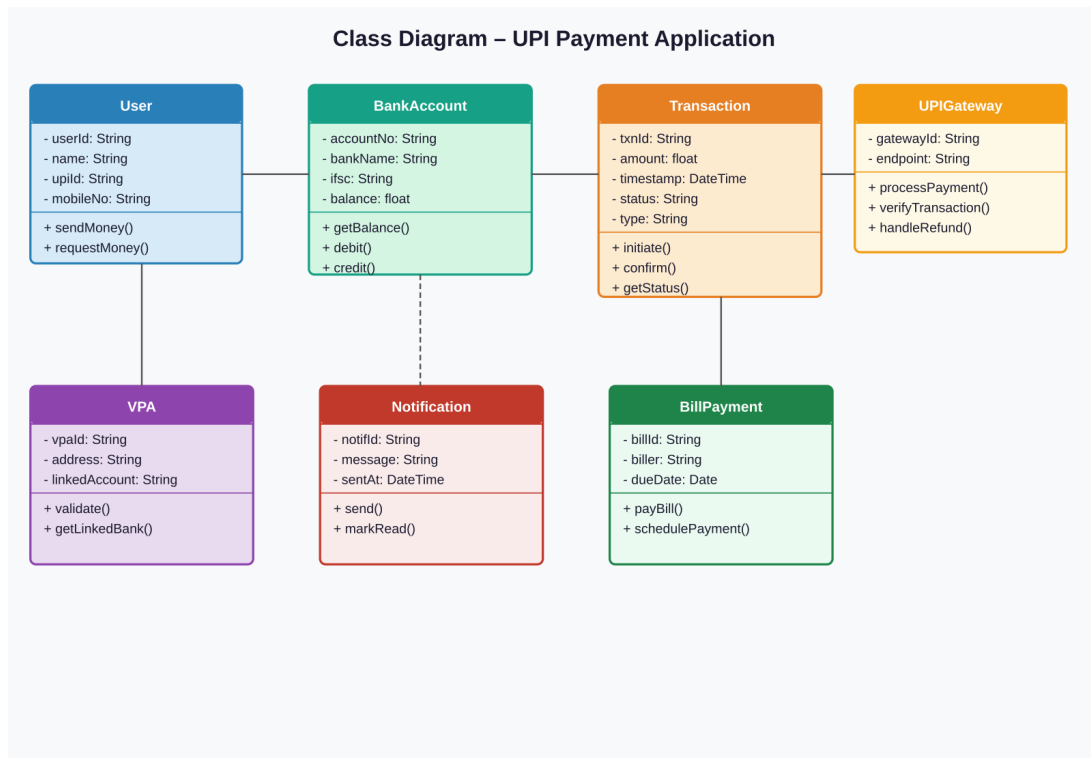


Figure 7.1 - Class Diagram: UPI Payment Application

Core Classes	User, BankAccount, Transaction, VPA, Notification, BillPayment, UPIGateway
Relationships	User has BankAccount (1..*); User initiates Transaction; Transaction via UPIGateway; VPA linked to BankAccount
Key Methods	sendMoney(), requestMoney(), debit(), credit(), validateVPA(), processPayment(), verifyTransaction(), payBill()

7.2 Sequence Diagram

End-to-end fund transfer: UPI details > VPA validation > MPIN > bank debit/credit > txn success > notifications.

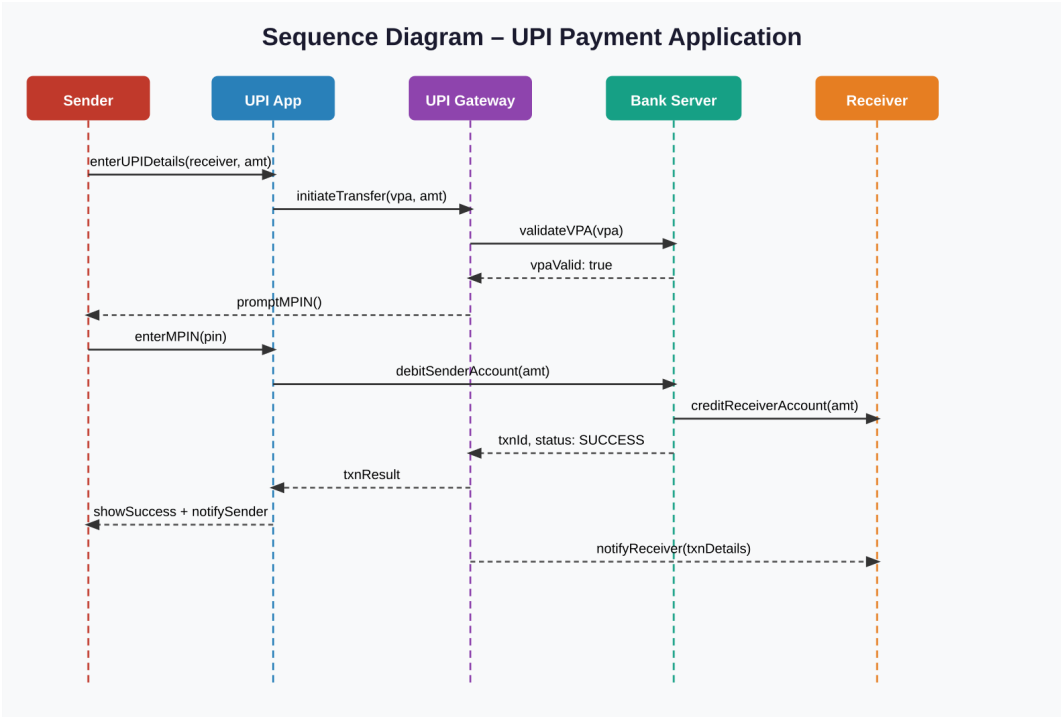


Figure 7.2 - Sequence Diagram: UPI Payment Application

Lifelines	Sender, UPI App, UPI Gateway, Bank Server, Receiver
Key Interactions	enterUPIDetails > initiateTransfer > validateVPA > promptMPIN > debitSender > creditReceiver > notifySender > notifyReceiver
Security	MPIN-based 2FA required before debit; unique txnId on success

8. Online Shopping Portal

An e-commerce platform supporting product browsing, cart management, order placement, online payment, order tracking, and returns/refunds, with admin product and order management.

8.1 Use Case Diagram

Customer and Admin actors interact with a feature-rich system covering the complete shopping lifecycle.

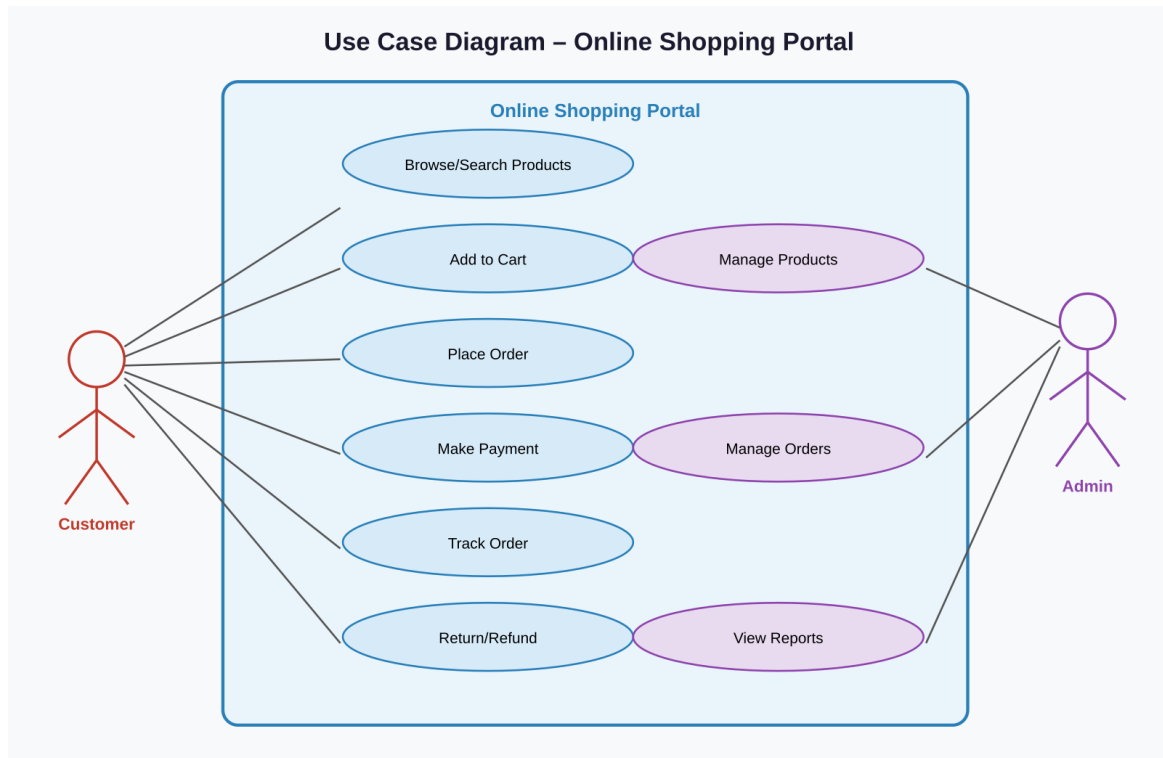


Figure 8.1 - Use Case Diagram: Online Shopping Portal

Actors	Customer, Admin
Customer Use Cases	Browse/Search Products, Add to Cart, Place Order, Make Payment, Track Order, Return/Refund
Admin Use Cases	Manage Products, Manage Orders, View Reports

8.2 Deployment Diagram

Six-node deployment: client devices, CDN, web server, application server, payment gateway, database cluster (MySQL + MongoDB + Redis), shipping API.

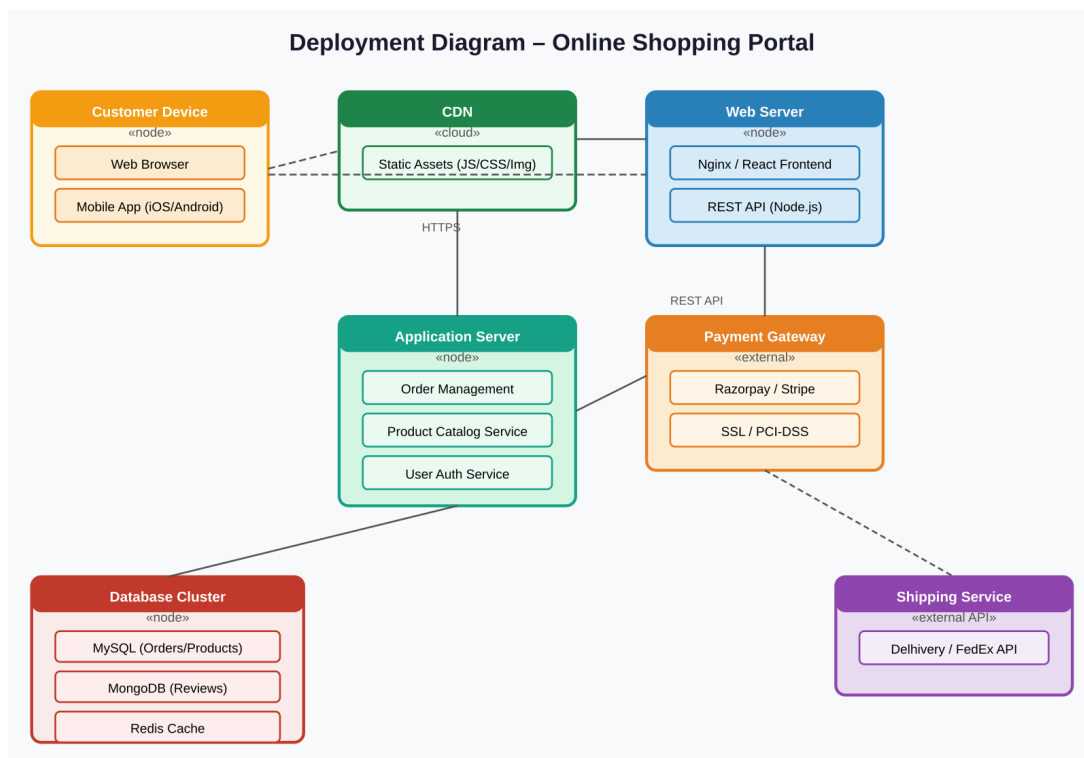


Figure 8.2 - Deployment Diagram: Online Shopping Portal

Nodes	Customer Device, CDN, Web Server (Nginx + Node.js REST API), Application Server, Payment Gateway, Database Cluster, Shipping Service API
Databases	MySQL (orders/products), MongoDB (reviews), Redis (cache)
Protocols	HTTPS (client-server), REST API (inter-service), external API (shipping/payment)

9. Courier Management System

A logistics platform handling end-to-end parcel lifecycle: booking, cost calculation, payment, label generation, delivery-agent assignment, real-time tracking, and delivery confirmation.

9.1 Activity Diagram

Step-by-step workflow from customer booking through payment verification, label generation, agent assignment, pickup, tracking updates, and delivery.

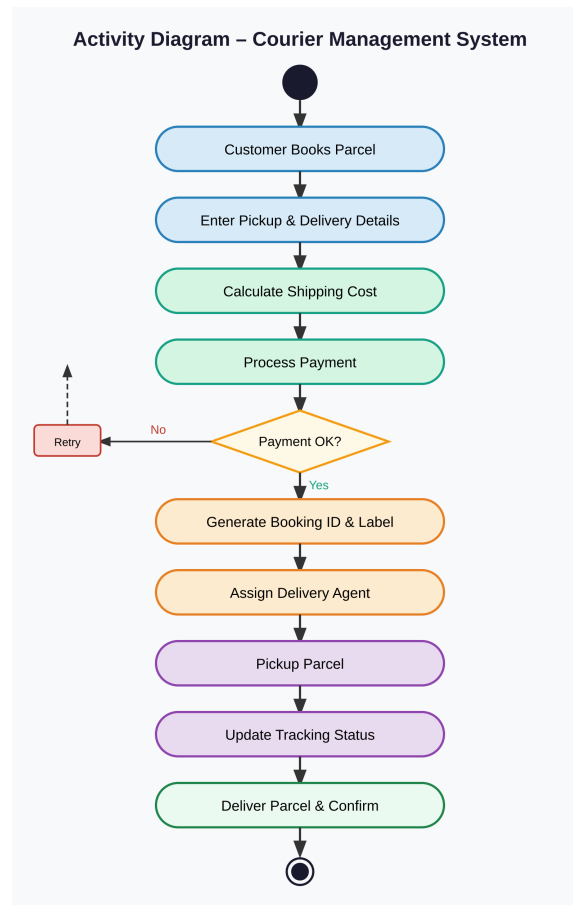


Figure 9.1 - Activity Diagram: Courier Management System

Decision Points	Payment OK?
Error Flow	Payment failure loops back to Retry
Key Activities	Book Parcel > Enter Details > Calculate Cost > Process Payment > Generate Label > Assign Agent > Pickup > Track > Deliver

9.2 Class Diagram

Seven classes covering Parcel, Booking, Customer, DeliveryAgent, TrackingRecord, Payment, and Warehouse.

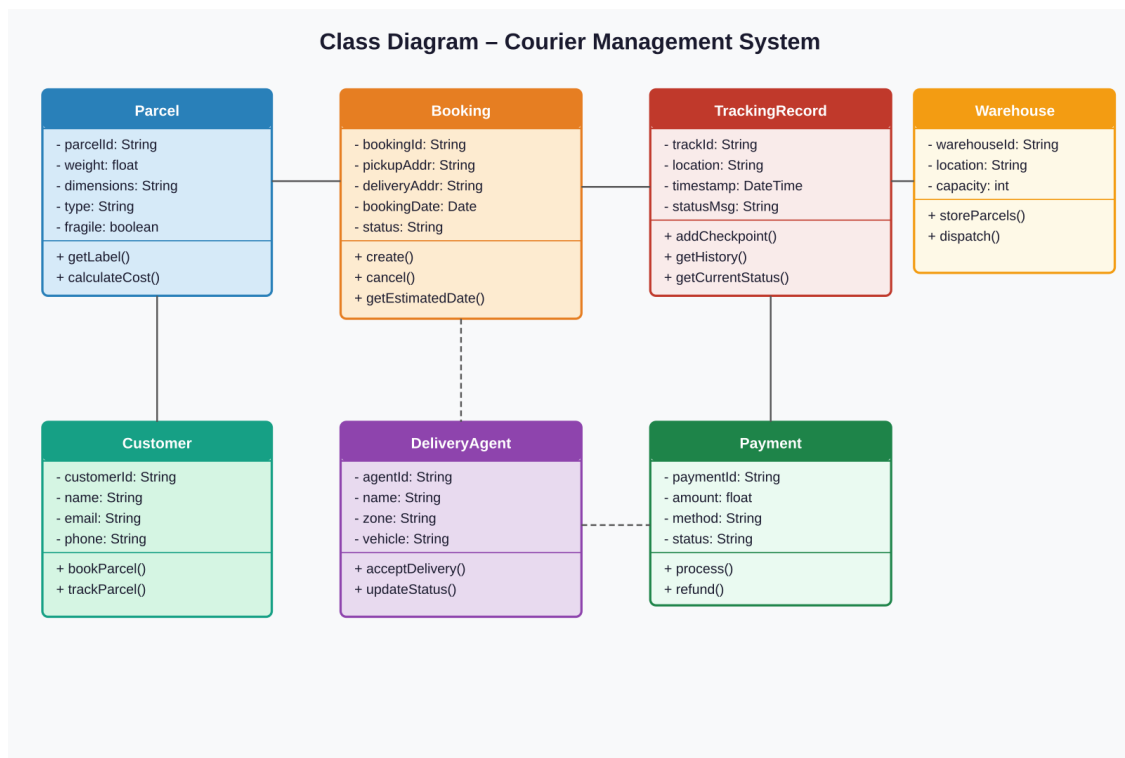


Figure 9.2 - Class Diagram: Courier Management System

Core Classes	Parcel, Booking, Customer, DeliveryAgent, TrackingRecord, Payment, Warehouse
Relationships	Customer creates Booking (1..*); Booking contains Parcel; DeliveryAgent handles Booking; TrackingRecord linked to Booking
Key Methods	calculateCost(), getLabel(), create(), cancel(), acceptDelivery(), updateStatus(), addCheckpoint(), process(), refund()

10. Tourism Management System

An online travel platform allowing tourists to search and book tour packages, process payments, manage bookings, and receive generated itineraries. Travel agents can create packages and manage bookings.

10.1 Use Case Diagram

Two actors - Tourist and Travel Agent - interact with the system supporting the full package booking and management lifecycle.

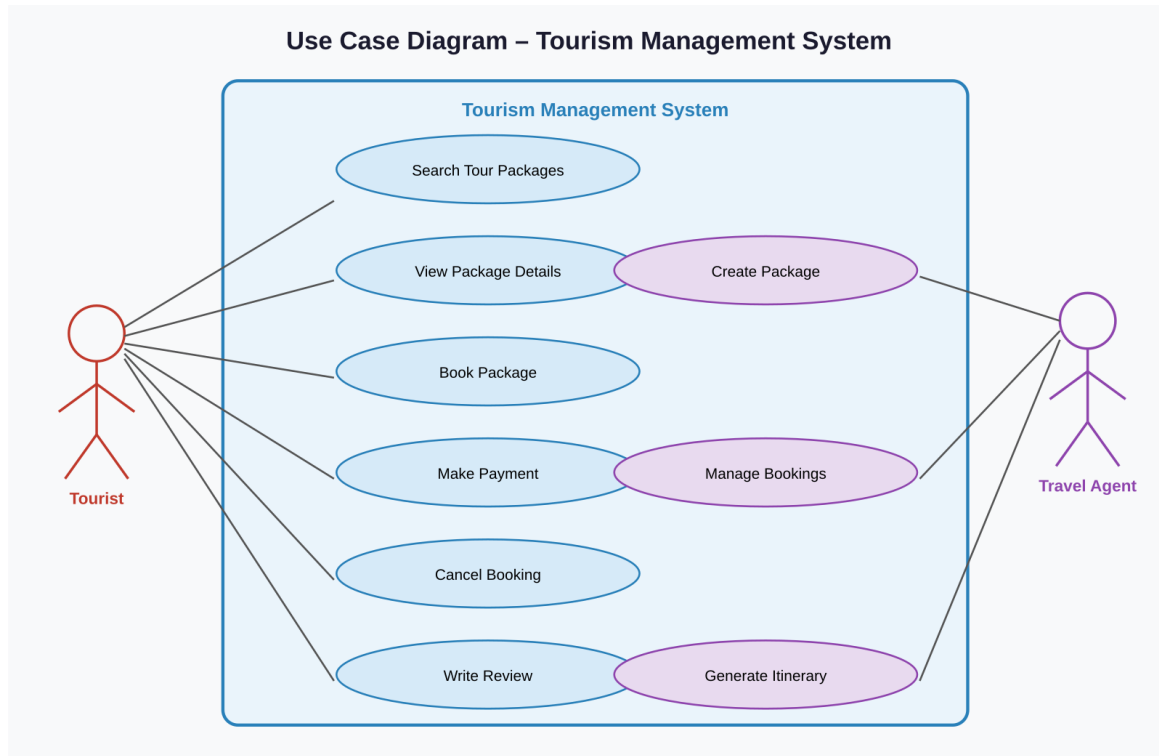


Figure 10.1 - Use Case Diagram: Tourism Management System

Actors	Tourist, Travel Agent
Tourist Use Cases	Search Tour Packages, View Package Details, Book Package, Make Payment, Cancel Booking, Write Review
Travel Agent Use Cases	Create Package, Manage Bookings, Generate Itinerary

10.2 Sequence Diagram

Full booking flow: searching packages > selecting > filling traveller details > payment processing > confirmation with itinerary via email/SMS.

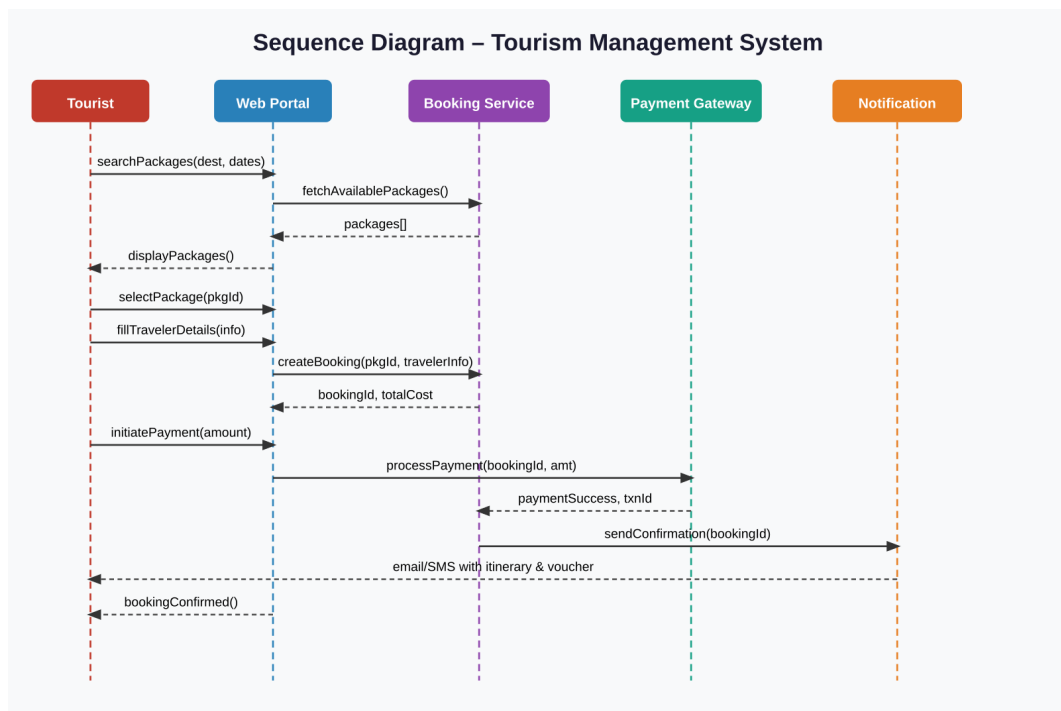


Figure 10.2 - Sequence Diagram: Tourism Management System

Lifelines	Tourist, Web Portal, Booking Service, Payment Gateway, Notification
Key Interactions	searchPackages > fetchAvailablePackages > selectPackage > fillTravelerDetails > createBooking > initiatePayment > processPayment > sendConfirmation > notifyTourist
Return Messages	packages[], bookingId/totalCost, paymentSuccess/txnId, email/SMS with itinerary and voucher